SILENT · 프로젝트 이해를 위한 코드 학습서

플레이 흐름을 코드로 끝까지 따라가기

파일·클래스 → 메서드 → 상태 → 이벤트 → 객체 수명

서버와 클라이언트를 따로 외우는 대신 플레이어의 한 행동을 양쪽 코드로 추적합니다. 기능마다 관계도와 실제 코드 발췌를 놓고 입력·출력·설계 의도·실패 처리를 설명합니다.



표지: 시작 화면 배경 원본. 실행 검증 캡처는 아닙니다.

> 앞선 공개용 기술문서는 경험 중심 요약입니다. 이 문서는 실제 메서드와 상태 소유자를 찾아가며 프로젝트를 공부하기 위한 코드 중심 동반 문서입니다.

2026-10-26 · Unity C# / Node.js TypeScript · 현재 로컬 코드·기존 문서 기준

목차와 학습 경로

큰 구조를 잡은 뒤 플레이 순서로 내려갑니다. 기능 관계도와 행 번호가 있는 원문 발췌를 함께 읽습니다.

학습 묶음	장	쪽
전체 구조·조립	0-2	3-5
시작 UI·입장·준비	3-6	6-9
클라 상태·요청·전송	7-9	10-12
서버 HTTP·참가·생존	10-12	13-15
데이터·관계·방 계획	13-15	16-18
월드·앵커·플레이어 생성	16-18	19-21
이동·잠금·손	19-21	22-24
대상·확대·퍼즐 입력	22-25	25-28
서버 판정·문·방 진행	26-28	29-31
소리·음성·조명	29-31	32-34
공동 엔딩·영상·결과	32-33	35-36
객체 수명·패턴	34-35	37-38
자동화·진단	36-37	39-40
색인·읽기 연습·Git	38-39	41-42

> 처음에는 0-2장으로 소유자를 익힌 뒤 3-12장에서 접속을 추적합니다. 이어 데이터·월드·상호작용을 읽고 39장의 연습으로 한 명령을 처음부터 끝까지 연결합니다.

0. 코드 옆에 놓고 읽는 방법

한 번의 입력이 어느 객체의 상태를 바꾸는지 추적합니다. 관계도는 호출 방향, 코드 발췌는 현재 구현의 근거입니다.

읽을 대상	뜻	실제 예
명령	앞으로 해 달라는 요청	JoinAsync·SubmitAnswerAsync
상태	객체가 소유하는 현재 사실	IsBusy·Phase·solved
이벤트	바뀐 결과를 통지	StateChanged·RoomLoaded
계약	소비자가 필요한 기능의 약속	IRoomAnswerService
어댑터	외부 API를 계약으로 변환	UnityWebRequest·Vivox
수명	생성부터 취소·해제까지	OnEnable·Dispose

0.1 한 기능을 읽는 다섯 단계

① 진입 메서드와 인자를 봅니다. ② 값과 Unity 참조를 구분합니다. ③ 누가 상태를 바꾸는지 찾습니다. ④ 저장 뒤 어떤 이벤트를 알리는지 봅니다. ⑤ 실패·취소·종료의 해제 책임을 찾습니다.

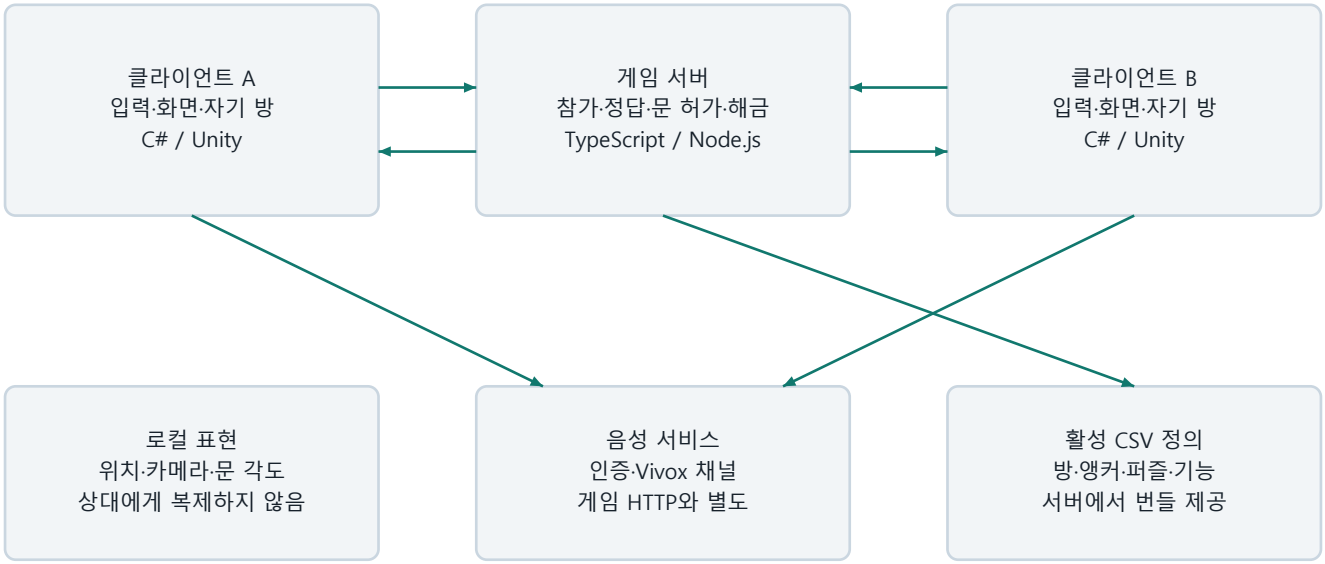
0.2 경로·행 번호·발췌 읽기

클라 경로는 Assets/3.Script, 서버 경로는 src 아래입니다. 발췌의 숫자는 작성 시점 실제 원문의 행 번호입니다. 전체 메서드가 아닌 부분은 시작·끝 범위로 표시하므로 주변 원문과 함께 읽습니다. 긴 줄은 PDF 안에서만 접어 표시합니다. 도식의 화살표는 호출 또는 상태·결과 전달입니다. 분기는 독립 소비자를, 연결선 없는 상자는 별도 실행 정책을 나타냅니다.

> 이번 작업은 코드 해설서 작성입니다. 게임 코드·CSV·씬을 수정하거나 전체 빌드를 다시 테스트하지 않습니다. 로컬 변경이 원격 저장소에 반영됐다는 뜻도 아닙니다.

1. 클라이언트·서버 전체 관계

두 PC의 표현과 서버의 진행 판정, 외부 음성 서비스를 먼저 나누면 같은 이름의 서비스들이 혼동되지 않습니다.



객체	소유 책임
GameEntryService	클라 입장·데이터·씬 전환 조정
SessionClient	클라의 세션 공개 계약과 연결 수명
SessionController	서버 HTTP 입력·출력
SessionService	서버 참가 검증과 게임 사용 사례
GameSession / RoomProgression	참가·준비 / 정답·허가·해금

입력 → 계약 → 전송 → 서버 Controller → Service → 규칙 객체 순서입니다. 응답을 클라 상태에 적용한 뒤 로컬 이벤트가 나갑니다. StateChanged나 RoomsChanged 이벤트 자체가 다른 PC로 전송되는 것은 아닙니다.

> 서버는 위치·문 각도·카메라를 복제하지 않습니다. 음성도 별도 Vivox 경로입니다. 서버 판정 권한과 정답 데이터의 비밀 보장은 다른 문제입니다.

2. 조립 지점: 누가 의존성을 연결하는가

생성자와 Initialize에 무엇을 넘기는지 보면 모듈의 실제 의존성과 소유자가 드러납니다.



호출과 책임

코드 경계	읽어야 할 동작
GameFlowBootstrapper.Awake	콘텐츠·세션·씬 전환 함수를 GameEntryService에 주입한다.
main.start	GameSession → ParticipantRegistry → SessionService → Controller를 조립한다.
종료 경계	씬 소유자와 프로세스 진입점이 자신이 만든 작업을 정리한다.

원문을 따라 읽기

서버 / main.ts : 28-34 · 현재 원문 발췌

```
28  const clock = new MonotonicMillisecondClock();
29  const participants = new ParticipantRegistry(clock, config.participantTimeoutMs);
30  const sessionService = new SessionService(session, participants, gameData);
31  const sessionController = new SessionController(sessionService);
32
33  const accessLogger = new HttpAccessLogger(undefined, token => {
34    const playerId = participants.getPlayerId(token);
```

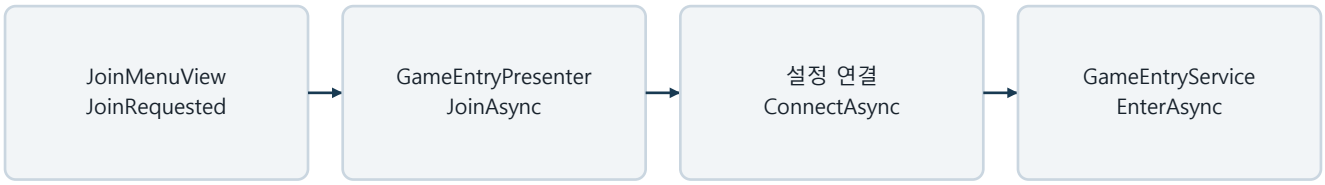
participants는 시계·만료 정책을, Service는 게임 상태·등록부·데이터를 받습니다. Controller는 상태를 직접 만들지 않습니다. new가 모인 곳이 명시적 의존성 주입의 실제 조립점입니다.

PeriodicTask는 listen 후 시작하고 종료 신호·서버 close에서 멈춥니다. DontDestroyOnLoad는 게임 소유자를 유지하는 엔진 처리이며 모든 기능을 전역 싱글턴으로 만드는 규칙은 아닙니다.

> 설계 의도 · 규칙이 환경 변수·씬 검색을 직접 수행하지 않게 합니다. 생성 순서와 실패 정리를 한 경계에서 추적합니다.

3. 시작 UI: 버튼을 입장 요청으로

버튼은 이벤트를 내고, Presenter는 작업을 시작하며, View는 바뀐 상태를 표시합니다.



호출과 책임

코드 경계	읽어야 할 동작
JoinMenuScreen.OnEnable	설정·View·Presenter·화면 취소 수명을 연결한다. 자동 접속을 호출하지 않는다.
GameEntryPresenter.Render	IsBusy와 참가 대기로 화면 표시를 계산한다.
OnDisable / Dispose	화면 요청 취소와 버튼·서비스 구독을 해제한다.

원문을 따라 읽기

클라 / World/GameEntryPresenter.cs : 27-32 · 현재 원문 발췌

```
27      connection = new ConfiguredServerConnection(settings, (address, ct) => service.EnterAsync(false, address, null, ct));
28      view.JoinRequested += Join;
29      view.LocalRequested += Local;
30      service.Changed += Render;
31      Render();
32  }
```

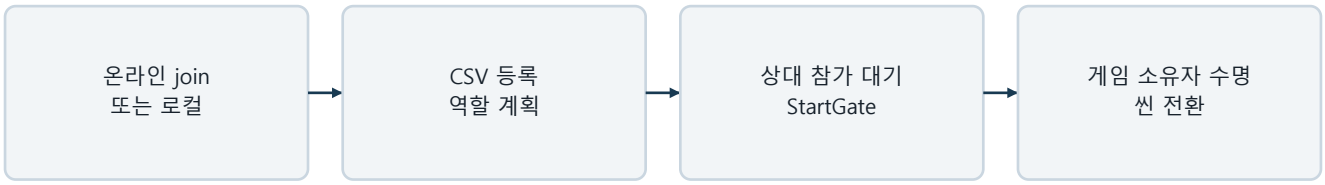
주입한 함수가 EnterAsync를 감쌉니다. View는 HTTP·토큰을 알지 못합니다. Changed는 접속 명령이 아니라 서비스 상태가 바뀌었으니 다시 표시하라는 알림입니다.

호환용 StartAsync가 존재해도 현재 JoinMenuScreen은 버튼 입력만 연결합니다. 같은 클래스가 제공하는 기능과 실제 화면에서 호출하는 기능을 구분합니다.

> 설계 의도 · UI 변경을 접속 규칙에서 분리합니다. 구독한 객체가 해제까지 맡아야 중복 요청·표시를 막습니다.

4. EnterAsync: 접속 이후까지의 작업

입장은 join 한 번이 아니라 데이터 검증·역할 선택·상대 대기·씬 전환·실패 보상의 전체 과정입니다.



호출과 책임

코드 경계	읽어야 할 동작
입력	useLocal·address·testRole·token. 온라인 역할은 서버 발급값이다.
상태	IsBusy·Data·Role·UnlockedRoomIds와 안내. 방 명령은 별도 위임한다.
실패	정의·허가를 초기화하고 이번 시도에서 새로 얻은 참가 자리를 정리한다.

원문을 따라 읽기

클라 / World/GameEntryService.cs : 177-188 · 현재 원문 발췌

```
177        // 커밋 경계: 로드 시작 전까지 화면 요청을 취소할 수 있습니다.
178        // 씬 전환이 시작되면 지속 게임 수명으로 넘겨 Join 화면 제거가 입장을 취소하지 않게 합니다.
179        token.ThrowIfCancellationRequested();
180        ownerToken.ThrowIfCancellationRequested();
181        await enterScene(ownerToken);
182        ownerToken.ThrowIfCancellationRequested();
183        entered = true;
184    }
185    catch (OperationCanceledException)
186    {
187        Message = "입장이 취소되었습니다.";
188    }
```

씬 전환 전까지 화면 요청을 취소할 수 있지만 실제 로딩은 ownerToken으로 이어갑니다. Join 화면 제거가 이미 시작한 Ingame 생성을 취소하지 않게 하는 경계입니다.

서버 데이터 실패를 로컬 CSV로 몰래 대체하지 않습니다. 그렇지 않으면 화면의 콘텐츠와 서버가 판정하는 정의가 다를 수 있습니다. entered·joinedHere는 성공과 보상 범위를 드러냅니다.

> 설계 의도 · 입장 절차를 조정하되 정답 규칙·뷰 생성을 직접 소유하지 않습니다. 취소 토큰의 소유권을 씬 전환 시 명시합니다.

5. StartGate: 입장 허용과 조작 허용

CanEnterScene은 두 명이 로딩해도 되는가, CanPlay는 두 ready가 끝나 조작해도 되는가입니다.



호출과 책임

코드 경계	읽어야 할 동작
Refresh	SessionInfo를 읽어 허용값·대기 안내를 계산한다. Unity·HTTP는 모른다.
WaitForParticipantsAsync	구독 후 즉시 검사하고 완료 직후 세션·참가 구성을 다시 확인한다.
로컬 모드	서버 대기는 우회하지만 데이터·플레이어 검사는 유지한다.

원문을 따라 읽기

클라 / World/SessionStartGate.cs : 54-61 · 현재 원문 발췌

```
54  bool enter = IsLocal || (current.IsJoined && current.ParticipantCount >= RequiredParticipants &&
55      (current.Phase == ClientSessionPhase.Loading || current.Phase == ClientSessionPhase.Playing));
56  bool play = IsLocal || (current.IsJoined && current.ParticipantCount >= RequiredParticipants && current.Phase ==
    ClientSessionPhase.Playing);
57  string message = play ? "" : !current.IsJoined ? "서버 연결이 종료되었습니다. 다시 입장해 주세요." :
58      !enter ? $"상대방 입장을 기다리고 있습니다. ({current.ParticipantCount}/{RequiredParticipants})" :
59      "두 참가자의 씬 준비 완료를 기다리고 있습니다.";
60  bool changed = state == null || state.SessionId != current.SessionId || state.IsJoined != current.IsJoined ||
61      CanEnterScene != enter || CanPlay != play || Message != message;
```

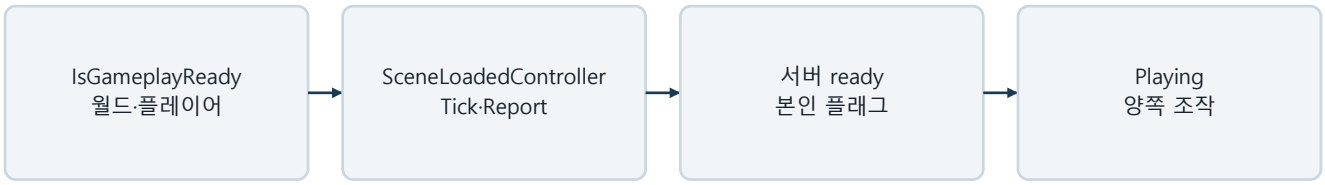
enter는 두 명과 Loading/Playing, play는 두 명과 Playing을 요구합니다. 두 번째 사용자가 대기 UI를 건너뛰어도 ready를 생략하는 것은 아닙니다.

TaskCompletionSource는 상태 이벤트를 기다리는 완료 약속입니다. 구독 직후 check로 사이에 지나간 상태를 보완하고 finally에서 구독을 해제합니다. 완료 직후 상대가 나간 경우도 재검사합니다.

> 설계 의도 · 허용 조건을 일반 C# 한곳에 모아 여러 화면에서 같은 인원·단계 if문을 복제하지 않습니다.

6. Ready: 무엇이 준비됐다고 보고하는가

방이 보이는 것과 이동·시점·상호작용까지 사용할 수 있는 것은 다릅니다.



호출과 책임

코드 경계	읽어야 할 동작
GameFlowBootstrapper	활성 Ingame·월드·플레이어의 이동·시점·상호작용을 확인한다.
SceneLoadedController	대기·보고·재시도·완료를 추적하고 본인 응답인지 확인한다.
SessionGameplayDriver	서버 Playing을 공통 PlayerControlState에 반영한다.

원문을 따라 읽기

클라 / Session/SessionSceneLoadedController.cs : 65-72 · 현재 원문 발췌

```
65      }
66      if (!retryInterval.TryConsume())
67          return;
68      Phase = SceneLoadedReportPhase.Reporting;
69      _ = ReportAsync(token);
70  }
71  private async Task ReportAsync(CancellationToken token)
72  {
```

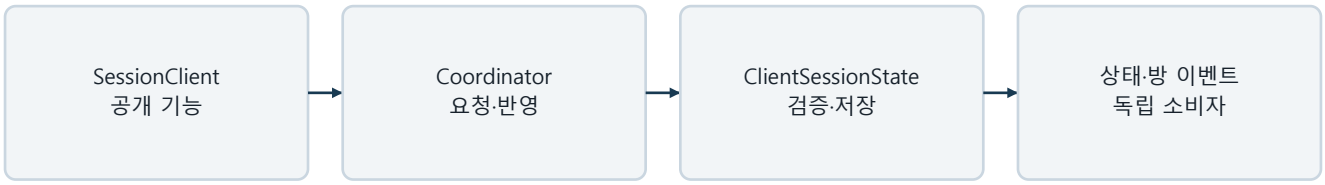
Tick이 프레임마다 호출돼도 retryInterval이 보고 간격을 제한합니다. Reporting 동안에는 같은 보고를 중복 시작하지 않습니다.

ReportAsync는 Busy·취소와 재시도 가능 오류, 종료할 실패를 구분합니다. 정상 응답에서 준비가 확인돼야 Complete와 ReportConfirmed로 넘어갑니다.

> 설계 의도 · 명시적 상태와 Tick으로 엔진 준비와 네트워크 보고의 시점을 나눠 추적합니다.

7. 세션 상태: 저장 뒤 이벤트를 알린다

SessionClient는 기능 창구, ClientSessionState는 내부 자격과 응답 상태의 소유자입니다.



호출과 책임

코드 경계	읽어야 할 동작
ApplyJoin	세션·본인·역할·토큰을 검증해 내부에 저장한다.
ApplySnapshot	본인·중복·단계·해금 ID의 정합성을 먼저 확인한다.
이벤트	SnapshotApplied → 변경 시 RoomsChanged → 변경 시 PhaseChanged 순서다.

원문을 따라 읽기

클라 / Session/ClientSessionState.cs : 144-155 · 현재 원문 발췌

```
144  var prevPhase = Phase;
145  Phase = nextPhase;
146  IsSceneLoaded = localPlayer.SceneLoaded;
147  ParticipantCount = snapshot.Players.Count;
148
149  SnapshotApplied?.Invoke();
150  if (snapshot.UnlockedRoomIds != null && (lastUnlockedRooms == null || !new
     HashSet<int>(lastUnlockedRooms).SetEquals(snapshot.UnlockedRoomIds)))
151  {
152      lastUnlockedRooms = snapshot.UnlockedRoomIds.ToArray();
153      RoomsChanged?.Invoke(Array.AsReadOnly(lastUnlockedRooms));
154  }
155
```

Phase·준비·인원을 먼저 반영하므로 이벤트 소비자는 새 상태를 읽습니다. 해금은 순서가 아닌 집합 비교로 달라졌는지 판단합니다.

두 명 미만인데 Playing이거나 본인 역할이 바뀐 응답은 정상 상태로 적용하지 않습니다. 참가 토큰은 내부에 두고 공개 SessionInfo에서 제외합니다.

> 설계 의도 · 가변 상태의 소유자와 공개 계약을 분리하고 알림 순서를 명시합니다.

8. 요청 실행기: 어떤 중복을 허용하는가

일반 요청·대기 명령·생존 확인·퇴장은 서로 다른 실행 정책입니다.



호출과 책임

코드 경계	읽어야 할 동작
RunAsync	일반 작업이 있으면 Busy.
RunAfterCurrentAsync	현재 일반 작업 뒤 대기 명령 하나. 무제한 FIFO가 아니다.
RunHeartbeatAsync / Exclusive	생존 확인은 별도 경로, 종료는 정리 후 독점.

원문을 따라 읽기

클라 / Session/SessionRequestRunner.cs : 65-71 · 현재 원문 발췌

```
65  public async Task RunHeartbeatAsync(Func<CancellationToken, Task> action, CancellationToken cancellationToken)
66  {
67      ValidateAction(action, cancellationToken);
68      if (heartbeatRunning || exclusivePending)
69          throw new SessionResultException(SessionFailureCode.Busy);
70      await ExecuteAsync(action, cancellationToken, true);
71  }
```

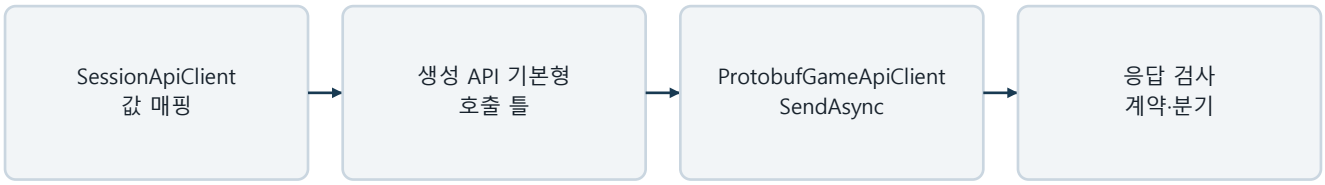
heartbeatRunning을 분리해 긴 데이터 요청 때문에 생존 확인까지 막히는 상황을 줄입니다. 같은 생존 요청의 중복은 여전히 제한합니다.

Coordinator는 정답·데이터를 대기 명령으로, leave를 독점 경로로 실행합니다. SessionClient의 연결 버전·취소는 이전 작업이 새 접속에 반영되는 것을 추가로 차단합니다.

> 설계 의도 · 요청 순서를 별도 객체로 모읍니다. 이는 영구 큐나 서버 재전송 시스템이 아닙니다.

9. 전송: C# 값이 HTTP가 되는 곳

생성 경로, 값 변환, 전송 오류 변환과 실제 Unity 요청의 책임을 나눠 읽습니다.



호출과 책임

코드 경계	읽어야 할 동작
HttpSessionGateway	전송 실패를 세션 실패 코드로 변환한다.
SendAsync	헤더·직렬화·제한 시간·취소·파싱·해제를 소유한다.
ResponseValidator	HTTP·미디어 타입·Protobuf 결과 분기와 요청 계약을 검사한다.

원문을 따라 읽기

클라 / Networking/ProtobufGameApiClient.cs : 37-47 · 현재 원문 발췌

```
37  var pending = request.SendWebRequest();
38  while (!pending.isDone)
39  {
40      if (token.IsCancellationRequested)
41      {
42          request.Abort();
43          token.ThrowIfCancellationRequested();
44      }
45      await Task.Yield();
46  }
47  token.ThrowIfCancellationRequested();
```

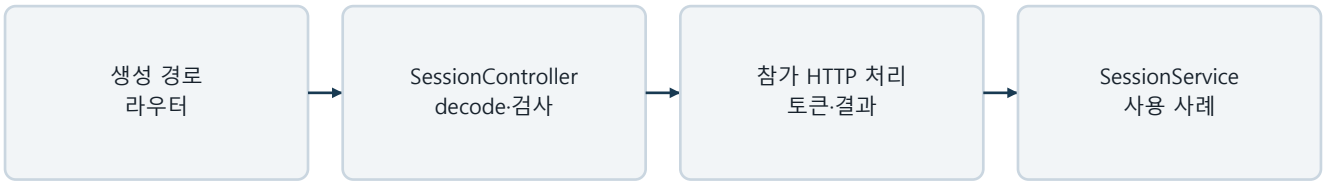
취소되면 Abort하고 완료 뒤에도 토큰을 확인합니다. using으로 요청 자원을 해제합니다. while은 HTTP를 계산하는 루프가 아니라 완료를 기다리는 비동기 경계입니다.

본문은 바이너리 Protobuf, 응답 최대 크기는 4MiB입니다. 일반 제한 10초와 별도 heartbeat 제한을 구분합니다. Protobuf는 암호화나 gRPC를 뜻하지 않습니다.

> 설계 의도 · 생성 기본형의 호출 틀과 SendAsync override는 템플릿 메서드·전송 어댑터의 결합입니다.

10. 서버 HTTP: 판정 전에 요청을 해석한다

HTTP 형식 검사와 게임 규칙 검사는 다른 계층의 책임입니다.



호출과 책임

코드 경계	읽어야 할 동작
createSessionHttpServer	OpenAPI 생성 경로로 분류하고 로그 관찰을 시작한다.
SessionController.room	roomNo·답 개수 등 본문 형태를 검사한다.
ParticipantHttpHandler	헤더와 결과를 Protobuf 응답으로 변환한다.

원문을 따라 읽기

서버 / controllers/SessionController.ts : 33-40 · 현재 원문 발췌

```
33      ProtobufHttp.validateAnswers(command.answers);
34      ParticipantHttpHandler.handle(request, response,
35        token => this.sessions.decideRoom(sessionId, token, command.roomNo, answer ? command.answers : undefined), "room");
36    } catch (error) {
37      this.contentFailure(request, response, error);
38    }
39  }
40  private contentFailure(request: IncomingMessage, response: ServerResponse, error: unknown): void {
```

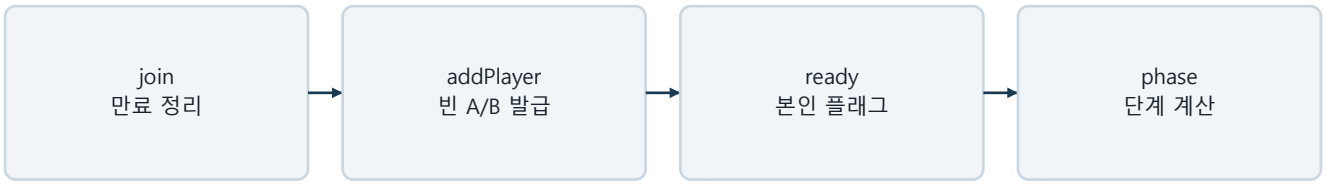
정답 명령이면 answers, 문 확인이면 undefined를 넘겨 decideRoom의 참가·역할 검증을 공유합니다. Controller는 solved 집합을 직접 변경하지 않습니다.

잘못된 본문은 ProtobufRequestError로, 예상하지 못한 오류는 내부 실패로 변환합니다. /health·/swagger/는 이 게임 명령 경로와 목적이 다릅니다.

> 설계 의도 · HTTP를 교체해도 게임 규칙이 요청 객체·응답 객체에 묶이지 않게 합니다.

11. 서버 세션: 인원·ready에서 단계를 계산한다

GameSession은 참가 배열과 ready를 소유하며 phase는 이 상태에서 유도합니다.



호출과 책임

코드 경계	읽어야 할 동작
SessionService.join	토큰 등록 실패 시 새 참가자를 되돌린다. 빈 세션의 새 시작에서 진행을 초기화한다.
ready / query	세션·참가 토큰을 검증한 뒤 준비 변경 또는 상태를 반환한다.
GameSession.getSnapshot	플레이어 객체를 복사해 외부가 내부 배열을 바꾸지 않게 한다.

원문을 따라 읽기

서버 / models/GameSession.ts : 32-41 · 현재 원문 발췌

```
32  get phase(): SessionPhase {
33    if (this.players.length < this.definition.roles.length) {
34      return "Waiting";
35    }
36
37    return this.players.every(player => player.ready)
38      ? "Playing"
39      : "Loading";
40  }
41
```

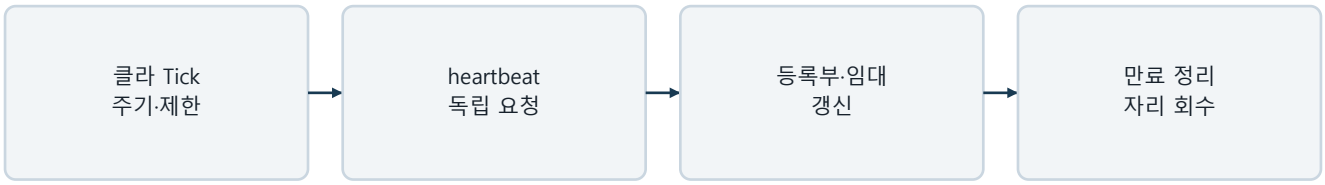
2명 미만은 Waiting, 정원은 찼지만 준비 미완료면 Loading, 모두 ready면 Playing입니다. 별도 phase 변수를 임의로 갱신하는 구조가 아닙니다.

이 단계 모델은 GoF State 클래스 계층이 아닙니다. 현재는 프로세스당 메모리 세션 하나이고, 팀 교체·재접속·영구 복구를 완성한 정책은 아닙니다.

> 설계 의도 · 참가·준비와 정답·해금을 분리하고 역할 발급 권한을 서버에 둡니다.

12. 생존: 조회와 임대 갱신은 다르다

클라 연결 감시와 서버 만료는 각자 시계와 책임을 갖습니다.



호출과 책임

코드 경계	읽어야 할 동작
HeartbeatController	sending으로 중복을 막고 확인 응답이 오래 없으면 Disconnect한다.
ParticipantRegistry	토큰·플레이어·HeartbeatLease를 연결한다. 만료된 임대는 살리지 않는다.
PollingController	대기 약 1초·진행 약 5초 조회. 같은 단계 응답마다 주기를 초기화하지 않는다.

원문을 따라 읽기

클라 / Session/Heartbeat/SessionHeartbeatController.cs : 46-55 · 현재 원문 발췌

```
46      if (clock.ElapsedMilliseconds - lastConfirmedMs >= settings.ConnectionTimeoutMs)
47      {
48          service.Disconnect(SessionFailureCode.ConnectionTimedOut);
49          return;
50      }
51      if (sending || !interval.TryConsume())
52          return;
53      sending = true;
54      _ = SendAsync(playerId, token);
55  }
```

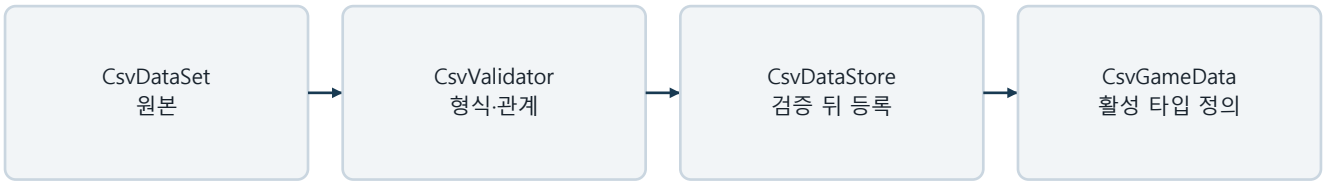
클라는 마지막 성공 heartbeat를 기준으로 단절을 감시합니다. 서버는 자체 단조 시계로 참가 기본 만료 10초를 판단하고 요청 진입과 주기 작업에서 정리합니다.

기본 heartbeat 약 3초와 서버 만료 검사 0.5초는 서로 다른 정책입니다. 진행 조회가 성공한다고 생존 임대 갱신까지 성공했다는 뜻은 아닙니다.

> 설계 의도 · 단조 시계·임대 규칙을 타이머 어댑터에서 분리해 검사합니다. 정수 시간이 네트워크 지연을 없애지는 않습니다.

13. CSV 등록: 성공한 새 정의만 교체한다

읽기·검증·등록·타입 변환과 플레이 중 상태는 다른 책임입니다.



호출과 책임

코드 경계	읽어야 할 동작
TryRegister	검증·취소 확인 뒤 Data를 바꾸고 캐시를 비운다.
MapTable / GetTable	표 이름·타입을 키로 변환 결과를 재사용한다.
번들	서버가 활성 표를 정렬·직렬화·해시해 메모리 결과로 재사용한다.

원문을 따라 읽기

클라 / Data/CsvDataStore.cs : 74-79 · 현재 원문 발췌

```
74      Data = data;
75      Validation = report;
76      mappedTables.Clear(); // 새 원본이 등록되면 이전 원본의 변환 캐시는 더 이상 사용하지 않습니다.
77      Registered?.Invoke(data); // 상태가 모두 반영된 후 알립니다.
78      return true;
79  }
```

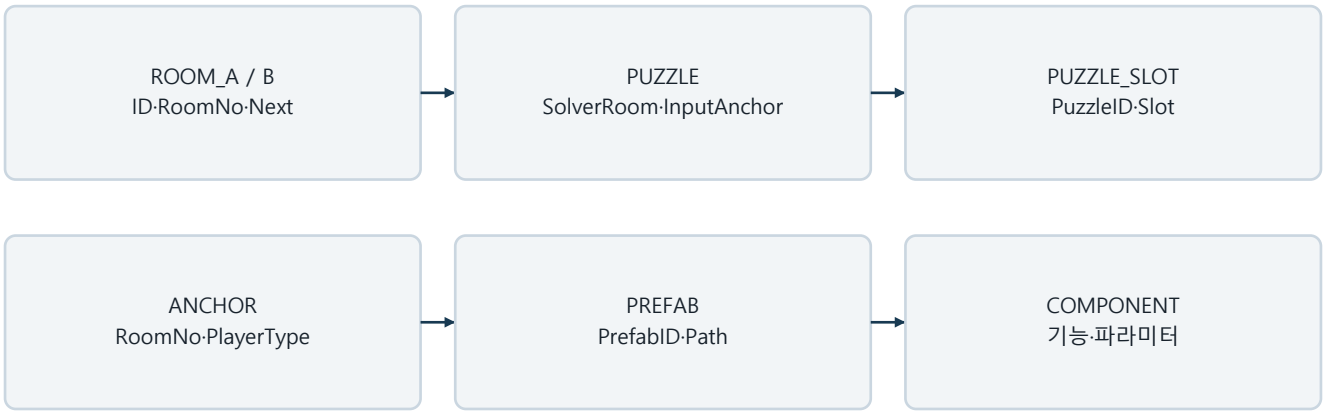
이 대입 전에 Validate와 IsValid 검사가 끝납니다. 새 원본을 반영한 뒤 변환 캐시를 비우고 Registered를 알리므로 소비자는 새 정의를 읽습니다.

실제 EnterAsync는 새 Store의 TryRegister를 사용합니다. InitializeAsync의 중복 초기화 지원 기능이 존재해도 모든 입장이 그 경로로 실행된다고 읽지 않습니다.

> 설계 의도 · 실패한 원본으로 정상 데이터를 덮지 않습니다. 데이터 정의에 씬 Transform과 가변 진행을 섞지 않습니다.

14. 데이터 관계: 키의 방향으로 읽는다

표 이름을 외우기보다 행 ID·방 번호·앵커 이름이 각각 무엇을 찾는지 구별합니다.



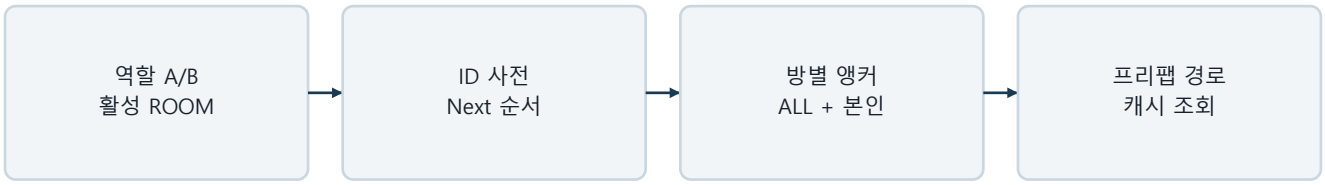
연결	의미
ROOM.ID / RoomNo	A의 1001과 B의 1002는 1번 방의 서로 다른 행이다. RoomNo와 ID는 다르다.
ROOM.Next	A 1001→1003, B 1002→1004. 다음 방 번호가 아니라 다음 행 ID다.
PUZZLE.SolverRoom / ClueRoom	정답을 입력하는 방과 단서가 있는 방의 ROOM 행 ID다.
PUZZLE.InputAnchor	ANCHOR_INTERACT의 행 ID를 가리킨다. Transform 이름이 아니다.
ANCHOR.AnchorID / PrefabID	계층 이름과 프리팹 정의를 잇는다. RoomNo·A/B/ALL 조건을 함께 쓴다.
PREFAB.Components	COMPONENT 선언을 연결한다. 새 기능 구현 자체를 자동 작성하지 않는다.

> 현재 활성 표 14개. SOUND·SOUND_LINK는 미사용입니다. WIRE·ARROW 등의 확장 정의가 남아 있어도 현재 콘텐츠라는 뜻은 아닙니다. 활성 등록·행 참조·기능 구현이 모두 필요합니다.

데이터 원문: Assets/11.CSV의 REGISTRY·ROOM_A/B·PUZZLE·PUZZLE_SLOT·ANCHOR_INTERACT/NONINTERACT·PREFAB·COMPONENT. 정답 원문은 이 문서에 싣지 않았습니다.

15. 방 계획: 정의 조회를 입장에 묶는다

RoomLoadPlan은 역할별 방 순서·앵커·프리팹 경로를 일반 객체로 구성하고 조회 결과를 재사용합니다.



호출과 책임

코드 경계	읽어야 할 동작
First / OrderedRooms	시작 행 하나를 찾고 Next를 따라가며 순환·누락을 확인한다.
Anchors / BuildAnchors	방 번호별 첫 필터·정렬 결과를 캐시한다.
새 입장	다른 데이터 묶음이면 새 계획을 만든다. Transform 캐시가 아니다.

원문을 따라 읽기

클라 / World/RoomLoadPlan.cs : 77-85 · 현재 원문 발췌

```
77          throw new ArgumentNullException(nameof(room));
78      if (anchorsByRoomNo.TryGetValue(room.RoomNo, out var anchors))
79          return anchors;
80      anchors = BuildAnchors(room.RoomNo);
81      anchorsByRoomNo.Add(room.RoomNo, anchors);
82      return anchors;
83  }
84  // 캐시 적중 경로에서 LINQ 캡처 객체까지 생성되지 않도록 구성 경로를 분리합니다.
85  private IReadOnlyList<AnchorData> BuildAnchors(int roomNo) => Array.AsReadOnly(data.InteractAnchors.Cast<AnchorData>().Con
    cat(data.NonInteractAnchors)
```

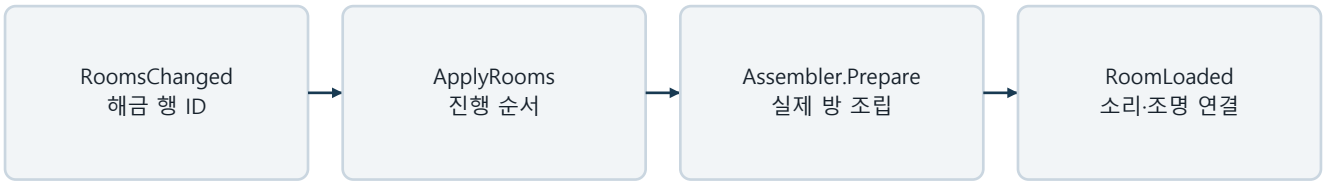
적중 경로는 저장 목록을 바로 반환하고, 실패 때만 BuildAnchors에서 활성·역할 필터와 정렬을 수행합니다. 반복 LINQ 캡처·배열 생성을 줄이려는 분리입니다.

정의에 대한 캐시와 씬 객체의 수명은 다릅니다. 새 데이터를 기존 계획에 끼워 넣거나 파괴된 이전 방 Transform을 전역 보관하는 규격은 아닙니다.

> 설계 의도 · 반복 조회 비용을 줄이되 초기 구성 비용과 엔진 객체 수명을 구분합니다.

16. 월드: 해금을 생성 완료로 번역한다

해금 목록을 받는다고 매번 방을 다시 만들지 않습니다. loaded·failures가 생성 시도를 추적합니다.



호출과 책임

코드 경계	읽어야 할 동작
Initialize	소유자·데이터·역할 계획을 연결하고 현재 해금을 즉시 적용한다.
ApplyRooms	요청한 행 중 성공·실패 캐시에 없는 방만 생성한다.
RequireFirstRoom / RetryFailedRooms	첫 방 실패는 입장 중단. 후속 실패는 기록하고 명시적으로 재시도한다.

원문을 따라 읽기

클라 / World/IngameWorldDriver.cs : 111-118 · 현재 원문 발췌

```
111         RoomSceneAssembler.Prepare(view, row, plan, data, owner.Assets, externallyManagedPrefabIds, previous);
112         loaded.Add(id, view);
113         failures.Remove(id);
114         added = true;
115     }
116     catch (Exception error)
117     {
118         if (instantiated && view != null)
```

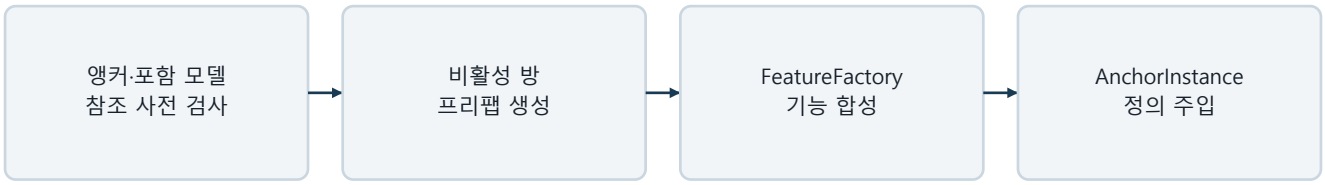
Prepare 성공 후 loaded에 넣고 RoomLoaded를 알립니다. 소리·조명은 준비되지 않은 방을 먼저 받지 않습니다.

실패한 후속 방을 매 조회마다 다시 만들지 않습니다. 이전 방은 유지하고 WorldReady는 첫 방이 loaded에 있는지로 계산합니다. 이는 플레이어 입력 준비와는 별도입니다.

> 설계 의도 · 데이터 진행과 엔진 생성의 경계를 둡니다. 생성 완료와 실패 이벤트를 구분합니다.

17. 방 내부: 앵커·프리팹·컴포넌트 조립

모든 참조를 먼저 확인하고, 비활성 상태에서 구성한 뒤 방을 활성화합니다.



호출과 책임

코드 경계	읽어야 할 동작
RoomSceneAssembler.Prepare	저장 배치 유지 또는 연결점 정렬 후 Load를 호출한다.
RoomSceneView.Load	성공 시 IsLoaded. 실패 시 생성물과 포함 모델 활성 상태를 정리한다.
AnchorFeatureFactory.Compose	알려진 기능만 붙이고 IAnchorDataReceiver에 정의를 주입한다.

원문을 따라 읽기

클라 / World/RoomSceneView.cs : 85-93 · 현재 원문 발췌

```
85        }
86        EscapeGame.Interaction.AnchorFeatureFactory.Compose(instance, operation.row, data);
87        var binding = instance.GetComponent<AnchorInstance>() ?? instance.AddComponent<AnchorInstance>();
88        binding.Initialize(operation.row, data);
89    }
90    foreach (var item in visibility)
91        item.target.SetActive(item.selected);
92    IsLoaded = true;
93    gameObject.SetActive(true);
```

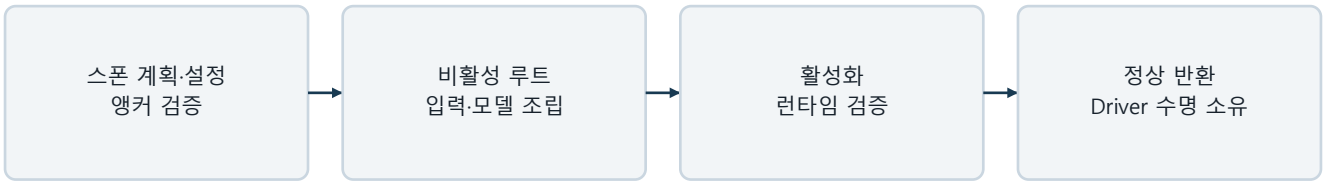
Compose는 임의 Script 문자열을 리플렉션 실행하는 기능이 아닙니다. 해당 팩토리는 Interaction/WorldInteractionTarget.cs 안에 선언돼 있습니다.

AnchorInstance.Initialize가 수신 계약을 구현한 자식 컴포넌트에 같은 정의·데이터를 전달합니다. 새 객체는 앵커의 로컬 원점·회전을 사용하고 포함 모델은 저장 경로로 찾습니다.

> 설계 의도 · 생성·기능 합성·정의 수신을 구분합니다. 앵커 누락을 임의 위치 생성으로 숨기지 않습니다.

18. 플레이어 생성: 활성화와 소유권 이관

팩토리는 플레이어 하나를 완성하거나 실패 자원을 정리하는 범위입니다.



호출과 책임

코드 경계	읽어야 할 동작
LocalPlayerFactory.Spawn	카메라·충돌·손·이동·시점·가림 표현을 조립한다.
PlayerSpawnDriver.Initialize	상호작용·소리·조명·측정 시계를 추가 연결한다.
실패 경계	정상 반환 전에는 pending 입력을 팩토리가 직접 추적한다.

원문을 따라 읽기

클라 / Player/LocalPlayerFactory.cs : 106-113 · 현재 원문 발췌

```
106        root.SetActive(true);
107        handAnimation?.ValidateRuntimeBinding();
108        // 정상 활성화 이후에는 각 입력 Driver가 수명을 소유합니다.
109        pendingMoveInput = null;
110        pendingPointerInput = null;
111        return rig;
112    }
113    catch
```

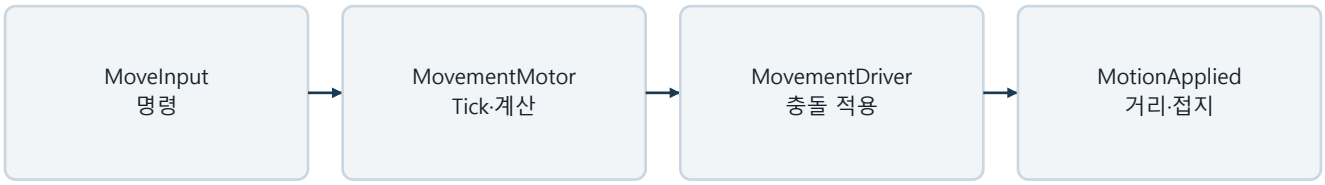
손 검증까지 성공한 뒤 pending 참조를 비워 각 Driver에 입력 정리 책임을 넘깁니다. catch에서는 비활성화·입력 Dispose·루트 Destroy를 수행합니다.

비활성 객체의 파괴 콜백에만 정리를 맡기지 않습니다. SpawnDriver의 후속 연결이 실패해도 플레이어를 정리하고 준비 완료로 진행하지 않습니다.

> 설계 의도 · 생성·활성화·검증을 같은 실패 처리에 둡니다. 팩토리는 상속보다 합성·수명 경계의 역할입니다.

19. 이동: 명령과 실제 결과를 분리한다

키 입력은 이동 의도이며 충돌 이후 거리와 접지가 표현의 기준입니다.



호출과 책임

코드 경계	읽어야 할 동작
Motor.Tick	정수 속도·가속·중력·나머지를 갱신해 mm 이동량을 반환한다.
Driver.Update	프레임 시간을 제한하고 Y 회전 기준으로 엔진 이동을 적용한다.
MotionApplied	실제 수평 거리(m)·접지를 손·보행 기능에 알린다.

원문을 따라 읽기

클라 / Player/PlayerMovementDriver.cs : 104-112 · 현재 원문 발췌

```
104     var previousPosition = transform.position;
105     var flags = controller.Move((horizontal + Vector3.up * upMm) / 1000f);
106     if ((flags & CollisionFlags.Below) != 0)
107         motor.OnGrounded();
108     var actual = transform.position - previousPosition;
109     actual.y = 0;
110     MotionApplied?.Invoke(actual.magnitude, grounded || (flags & CollisionFlags.Below) != 0);
111 }
112
```

이전 위치와 충돌 적용 후 위치 차이에서 y를 제거한 값이 실제 수평 이동입니다. 벽에 막히면 요청한 이동량과 달라집니다.

Motor는 1ms 적분·나머지 누적으로 프레임 분할 절삭 차이를 줄입니다. 정수 계산이 Unity 물리·입력까지 결정적인 리플레이를 보장하는 것은 아닙니다.

> 설계 의도 · 순수 계산과 엔진 적용을 분리하고 확정 이동 결과를 독립 표현이 소비하게 합니다.

20. 포인터와 조작 잠금의 공통 상태

커서·시점·UI가 각각 이동 허용 bool을 덮어쓰지 않게 합니다.



호출과 책임

코드 경계	읽어야 할 동작
PointerDriver.Update	UI 위 클릭·커서 해제·재획득·skipFrame을 처리한다.
ControlState	활성·포커스·일시정지·게임 허용·잠금 수를 합쳐 판단한다.
InputActionLease	원본 입력을 복제하여 기능별 활성·정리 수명을 갖는다.

원문을 따라 읽기

클라 / Player/PlayerControlState.cs : 66-73 · 현재 원문 발췌

```
66  public IDisposable BlockForUi()
67  {
68      uiBlocks++;
69      captured = false;
70      Changed?.Invoke();
71      return new UiBlock(this);
72  }
73  private sealed class UiBlock : IDisposable
```

화면은 잠금 자원을 받고 해제하면 자기 잠금만 줄입니다. 확대 화면이 닫혀도 엔딩·상대 대기 잠금이 남아 있으면 조작은 계속 막힙니다.

커서를 다시 잡는 클릭이 사물을 동시에 작동시키지 않도록 재획득에서 반환하고 다음 입력 프레임을 건너뜁니다. OnDisable은 자신이 소유한 이전 커서 상태를 복원합니다.

> 설계 의도 · 입력 캡처와 허용 규칙을 합성합니다. 네트워크·UI가 이동 Motor를 직접 변경하지 않습니다.

21. 손·보행: 확정 이동을 관찰한다

손 Speed·카메라 흔들림·발소리는 같은 이동 결과를 공유하되 각 상태는 따로 소유합니다.



호출과 책임

코드 경계	읽어야 할 동작
HandAnimationDriver	접지 거리/시간으로 속도를 적용하고 잠금·비활성화 때 0으로 만든다.
ValidateRuntimeBinding	활성화·초기화 후 실제 Float Speed를 확인한다.
GaitDriver	실제 이동과 설정으로 흔들림·Stepped 이벤트를 만든다.

원문을 따라 읽기

클라 / Player/PlayerHandAnimationDriver.cs : 61-70 · 현재 원문 발췌

```
61  if (!animator.isInitialized)
62      animator.Rebind();
63  if (!animator.isInitialized)
64      throw new InvalidOperationException($"손 애니메이터 {animator.name}의 초기화를 완료하지 못했습니다.");
65  bool found = false;
66  foreach (var parameter in animator.parameters)
67      if (parameter.nameHash == speedParameter && parameter.type == AnimatorControllerParameterType.Float)
68          found = true;
69  if (!found)
70      throw new InvalidOperationException($"손 애니메이터 {animator.name}에 실수형 {speedParameterName} 파라미터가 필요합니다.
");
```

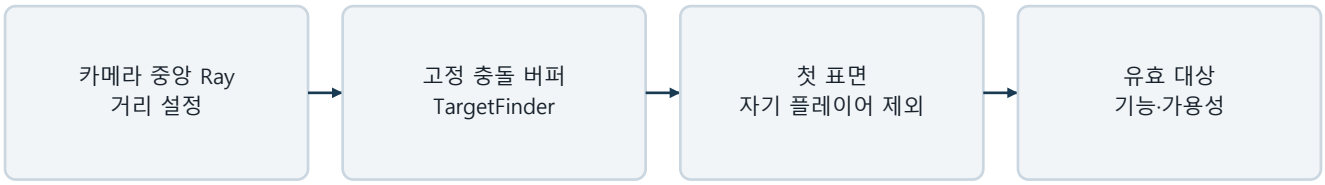
Rebind 후에도 초기화되지 않았으면 실패를 전달합니다. Factory가 활성화 직후 명시적으로 검사하며 매 프레임 목록을 다시 찾지 않습니다.

A/B는 공통 제어기의 대기·걷기를 사용합니다. 구독은 활성 수명에 맞춰 해제하고, 먼저 파괴된 Animator는 속도 출력 때 확인합니다. 비활성 초기화 순서 오류와 실제 설정 누락을 구분해야 합니다.

> 설계 의도 · 이동 규칙이 손·소리를 모르게 하고 표현 어댑터가 결과를 관찰합니다.

22. 대상 감지: 가장 가까운 표면을 본다

대상·가림·거리는 확대 UI를 만들기 전에 확인합니다.



호출과 책임

코드 경계	읽어야 할 동작
Find	가장 가까운 충돌체 부모에서 WorldInteractionTarget을 찾는다.
Target.Initialize	활성 앵커·기능 선언·색·유효 Collider를 확인한다.
InteractionDriver	대상이 바뀌면 하이라이트를 교체하고 클릭을 명령으로 바꾼다.

원문을 따라 읽기

클라 / Interaction/InteractionTargetFinder.cs : 15-23 · 현재 원문 발췌

```
15  int count = Physics.RaycastNonAlloc(ray, hits, distanceMm / 1000f, Physics.DefaultRaycastLayers, QueryTriggerInteraction.Ignore);
16  if (count == hits.Length)
17      return null;
18  Collider nearest = null;
19  float distance = float.MaxValue;
20  for (int i = 0; i < count; i++)
21      if (!hits[i].collider.transform.IsChildOf(player) && hits[i].distance < distance)
22      {
23          nearest = hits[i].collider;
```

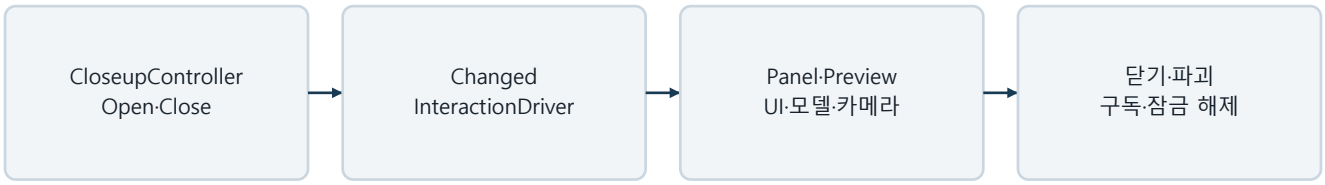
32칸 버퍼가 가득 차면 이번 선택을 거절합니다. 제한된 결과에서 잘못된 가까운 표면을 선택하는 것을 피하기 위한 정책입니다.

가장 가까운 표면이 대상이 아니면 그 뒤의 대상을 다시 고르지 않습니다. RaycastNonAlloc의 성능 선택과 벽 뒤 상호작용을 막는 규칙은 별개로 읽습니다.

> 설계 의도 · 할당을 줄이면서 버퍼 포화 시 행동도 명시합니다. 실제 Collider 배치는 통합 검사 대상입니다.

23. 확대 화면: 잠금과 미리보기 수명

일반 C# 확대 상태와 실제 UI·카메라·복제 모델을 분리합니다.



호출과 책임

코드 경계	읽어야 할 동작
Open	가용 대상·지원 기능을 검사하고 조작 잠금을 획득한다.
OnCloseupChanged	이전 답 UI·미리보기를 지우고 새 대상 표현을 구성한다.
Close / Dispose	대상·잠금 참조를 비우고 소유한 잠금을 해제한다.

원문을 따라 읽기

클라 / Interaction/CloseupController.cs : 44-51 · 현재 원문 발췌

```
44            return;
45        Target = null;
46        var owned = uiLock;
47        uiLock = null;
48        owned?.Dispose();
49        Changed?.Invoke();
50    }
51    public void Refresh()
```

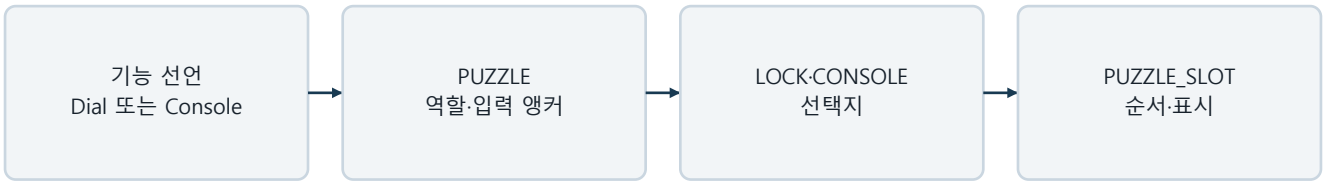
참조를 먼저 비우므로 반복 종료가 같은 잠금을 두 번 해제하지 않습니다. Changed 소비자는 닫힌 상태를 읽습니다.

PanelView와 Preview는 Canvas·렌더 텍스처·복제 모델을 정리합니다. C# 참조가 남아 있어도 엔진 객체는 먼저 파괴될 수 있어 Unity null 검사와 반복 Dispose 보호가 필요합니다.

> 설계 의도 · 상태·표현을 나누고 잠금 소유권은 Controller에 둡니다. 씬 종료와 늦은 답 응답까지 같이 추적합니다.

24. 입력 계획: 데이터가 선택지를 정한다

PuzzleInputPlan은 입력 정의이고 편집 중 선택값·성공 상태의 소유자는 아닙니다.



호출과 책임

코드 경계	읽어야 할 동작
Create	활성 앵커·역할·Solver 방·정확한 퍼즐 하나를 검사한다.
Choices	숫자·문자·색 다이얼 또는 버튼 선택지를 검증한다.
Slots	개수·1부터 이어지는 순서가 정의와 맞는지 검사한다.

원문을 따라 읽기

클라 / Interaction/PuzzleInputPlan.cs : 72-77 · 현재 원문 발췌

```
72  var slots = data.PuzzleSlots.Where(row => row.PuzzleId == puzzle.Id).OrderBy(row => row.Slot).ToArray();
73  if (puzzle.SlotCount < 1 || slots.Length != puzzle.SlotCount || !slots.Select(row => row.Slot).SequenceEqual(Enumerable.Range(1, slots.Length)))
74      throw new InvalidOperationException("PUZZLE_SLOT의 개수·순서가 PUZZLE.SlotCount와 일치해야 합니다.");
75  string[] choices;
76  if (dial)
77  {
```

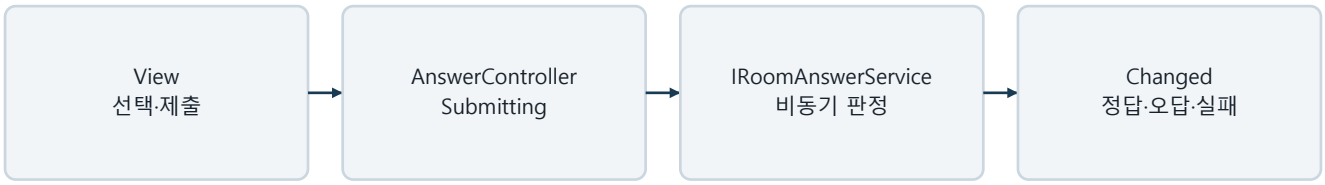
표가 틀린 상태에서 빈 버튼 일부만 만드는 대신, 입력 계획 생성 단계에서 실패시킵니다. 읽기 목록이 Controller에 전달됩니다.

입력 계획에는 정답 필드가 없지만 전체 활성 CSV에는 판정 정의가 포함됩니다. 계획에 정답을 저장하지 않는 것과 데이터 은닉은 다른 의미입니다.

> 설계 의도 · 선택지 정의와 가변 편집 상태를 분리합니다. 새 알고리즘은 별도 구현·검증을 추가합니다.

25. 답 UI: 값을 복사하고 결과를 기다린다

닫힌 화면에 늦은 응답을 적용하지 않도록 제출 상태·취소·표시를 함께 읽습니다.



호출과 책임

코드 경계	읽어야 할 동작
Cycle / Choose / Reset	선택지 안에서만 편집한다. 제출 중·정답 완료면 수정하지 않는다.
SubmitAsync	배열 복사본을 보내고 응답 뒤 토큰·disposed를 검사한다.
Phase / Failure	정상 오답과 전송·응답 실패를 구분한다.

원문을 따라 읽기

클라 / Interaction/PuzzleAnswerController.cs : 121-131 · 현재 원문 발췌

```
121  token.ThrowIfCancellationRequested();
122  var result = await service.SubmitAnswerAsync(Plan.RoomNo, (string[])values.Clone(), token);
123  token.ThrowIfCancellationRequested();
124  if (disposed)
125      return;
126  if (result == null)
127  {
128      Phase = AnswerInputPhase.Failed;
129      Failure = SessionFailureCode.InvalidResponse;
130  }
131  else if (!result.IsSuccess)
```

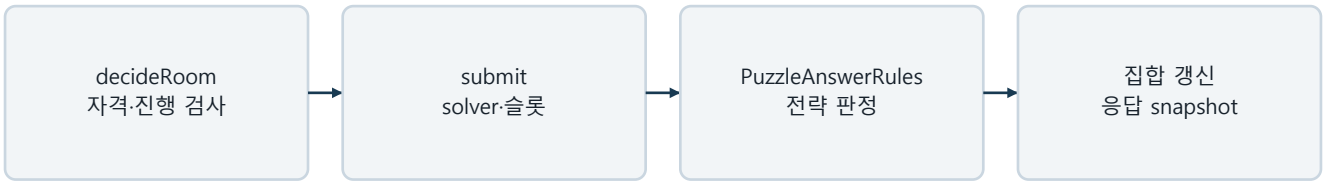
values.Clone은 화면이 쓰는 가변 배열을 비동기 요청과 공유하지 않기 위한 복사입니다. 닫힌 Controller는 결과와 Changed를 반영하지 않습니다.

정상 결과는 Correct/Incorrect, 잘못된 응답·연결 문제는 Failed입니다. Dispose는 취소와 이벤트 참조 해제를 수행하며 예외는 진단 경로로 보냅니다.

> 설계 의도 · UI는 좁은 서비스 계약만 알아도 됩니다. 테스트가 토큰·HTTP 생성에 직접 결합되지 않습니다.

26. 서버 정답: 규칙과 사용 사례를 나눈다

SessionService는 참가·단계·방을, RoomProgression은 정답·문·해금 집합을 소유합니다.



호출과 책임

코드 경계	읽어야 할 동작
decideRoom	서버가 인증한 역할로 방을 찾고 Playing을 검사한다.
submit	해금된 Solver 방과 슬롯을 확인한 뒤 정답이면 solved에 넣는다.
SlotAnswerRule	개수·순서·대문자 문자열 비교. 임의 수식·공백 제거 처리는 없다.

원문을 따라 읽기

서버 / models/RoomProgression.ts : 39-47 · 현재 원문 발췌

```
39    const slots = this.data.puzzleSlots.filter(slot => slot.puzzleId === puzzleId).sort((a, b) => a.slot - b.slot);
40    const correct = this.answerRules.evaluate(puzzle, slots, answers);
41    if (!correct) return { accepted: true, correct: false };
42    this.solved.add(puzzleId);
43    const room = rooms.find(room => room.id === puzzle.solverRoom)!;
44    this.checkDoor(role, room.roomNo);
45    return { accepted: true, correct: true };
46  }
47  findRoom(role: PlayerRole, roomNo: number): RoomData | undefined {
```

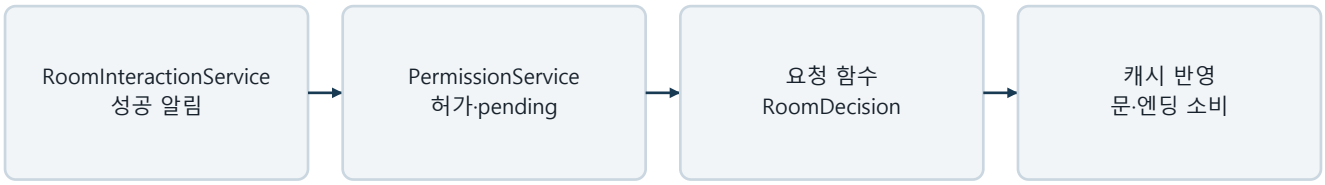
accepted=true·correct=false는 허용된 요청의 오답입니다. 요청 불허와 구분합니다. 정답이면 puzzleId를 solved에 넣고 checkDoor를 호출합니다.

Type별 IPuzzleAnswerRule 선택 경계가 있지만 현재 등록 종류는 같은 SlotAnswerRule을 공유합니다. 이름만 보고 각 퍼즐의 별도 물리·수식 알고리즘이 구현됐다고 읽지 않습니다.

> 설계 의도 · 인증·사용 사례·판정 전략을 분리합니다. 확장할 때 규칙 등록과 검사도 함께 추가합니다.

27. 문 허가 캐시: 세대로 늦은 응답을 막는다

한 입장에 누적되는 허가와 문 표현의 개폐 상태는 다릅니다.



호출과 책임

코드 경계	읽어야 할 동작
Submit / CheckDoor	답이 있으면 제출, 없으면 허가 확인. 허가 캐시가 있으면 재전송하지 않는다.
ExecuteAsync	같은 방 중복 요청과 잘못된 방·값의 응답을 거절한다.
Reset	세대를 올려 이전 응답을 무효화하고 캐시를 비운다.

원문을 따라 읽기

클라 / World/RoomPermissionService.cs : 50-60 · 현재 원문 발췌

```
50      token.ThrowIfCancellationRequested();
51      if (version != generation)
52          throw new OperationCanceledException("이전 입장의 문 판정입니다.");
53      if (decision == null || decision.RoomNo != roomNo || (answers == null && decision.Value != decision.DoorAllowed))
54          throw new SessionResultException(SessionFailureCode.InvalidResponse);
55      if (decision.DoorAllowed)
56          permitted.Add(roomNo);
57      return decision.Value;
58  }
59  finally
60  {
```

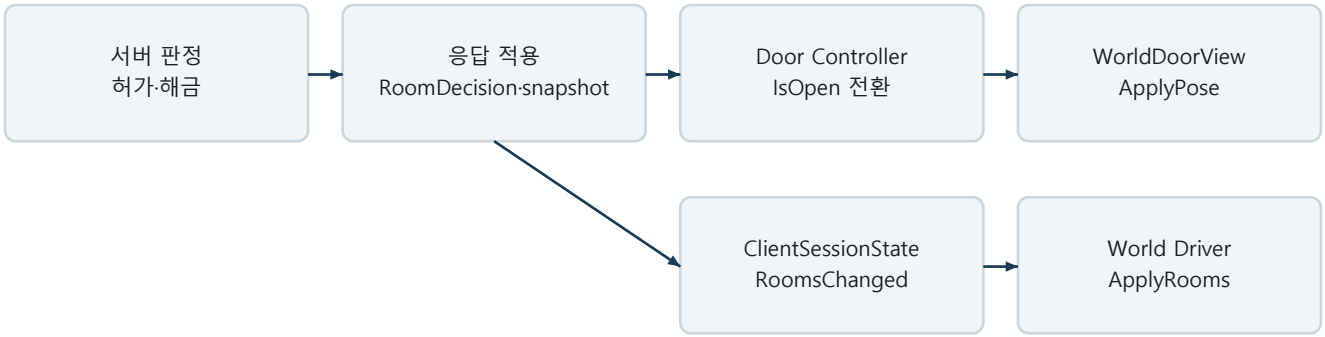
generation이 다르면 이전 입장 응답을 취소로 폐기합니다. DoorAllowed일 때만 캐시에 넣고, 문 확인 응답에서는 Value와 DoorAllowed가 같은지도 확인합니다.

RoomInteractionService는 자기 정답 성공 후 RoomSolved를 알립니다. 마지막 방에서는 정답 성공이어도 공동 문 허가는 없을 수 있어 두 사실을 구분해야 합니다.

> 설계 의도 · 모든 문이 같은 입장의 허가 사실을 공유하되 오래된 응답은 현재 캐시를 바꾸지 못하게 합니다.

28. 문·다음 방: 세 상태가 연결되는 곳

서버 permitted·unlocked, 문 IsOpen, 월드 loaded를 서로 다른 열에 놓고 읽습니다.



호출과 책임

코드 경계	읽어야 할 동작
ToggleAsync	첫 허가 후 개폐 상태를 바꾼다. IsBusy·취소로 중복을 제한한다.
ApplyPose	닫힌 자세에서 경첩 기준 위치·회전을 다시 계산하고 충돌 변환을 동기화한다.
checkDoor	자기/양쪽 정답 조건을 만족하면 허가와 다음 행 해금을 누적한다.

원문을 따라 읽기

클라 / Interaction/DoorInteractionController.cs : 46-54 · 현재 원문 발췌

```
46      bool decision = allowed || await service.CheckRoomDoorAsync(roomNo, request.Token);
47      request.Token.ThrowIfCancellationRequested();
48      if (!decision || disposed)
49          return false;
50      allowed = true;
51      IsOpen = !IsOpen;
52      Changed?.Invoke();
53      return true;
54  }
```

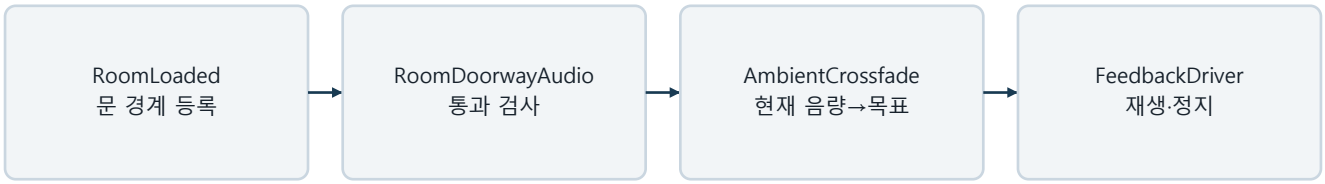
allowed는 허가, IsOpen은 개폐입니다. 허가된 문을 왕복 개폐할 때마다 서버에 다시 묻지 않습니다. 개폐를 상대 PC에 복제하지도 않습니다.

해금은 snapshot 적용 → RoomsChanged → ApplyRooms로 방 생성에 연결됩니다. 문 통과라는 실제 공간 이동은 별도 감지이며, 기존 방은 제거하지 않아 뒤를 볼 수 있습니다.

> 설계 의도 · 진행·문 표현·공간 로딩을 한 bool에 섞지 않습니다.

29. 소리: 경계와 이동 결과를 소비한다

발동 조건·음량 보간·AudioSource 출력을 나눠 읽습니다. 현재 소리는 CSV가 아닌 GameFeedback 에셋을 사용합니다.



호출과 책임

코드 경계	읽어야 할 동작
RegisterRoom	이미 등록한 방은 건너뛰고 열린 문 영역과 이전·현재 위치를 사용한다.
Select / Tick	약 3초 동안 음량을 보간한다. 역주행은 현재 음량부터 이어간다.
PlayCue	확대 UI 조건·쿨다운·순환 순서·음원 채널을 적용한다.

원문을 따라 읽기

클라 / Audio/AmbientCrossfade.cs : 30-41 · 현재 원문 발췌

```
30      target = channel;
31      elapsed = 0;
32      Array.Copy(gains, start, gains.Length);
33      if (immediate)
34      {
35          for (int i = 0; i < gains.Length; i++)
36              gains[i] = i == target ? 1 : 0;
37          Array.Copy(gains, start, gains.Length);
38          elapsed = duration;
39      }
40  }
41  public bool Tick(double seconds)
```

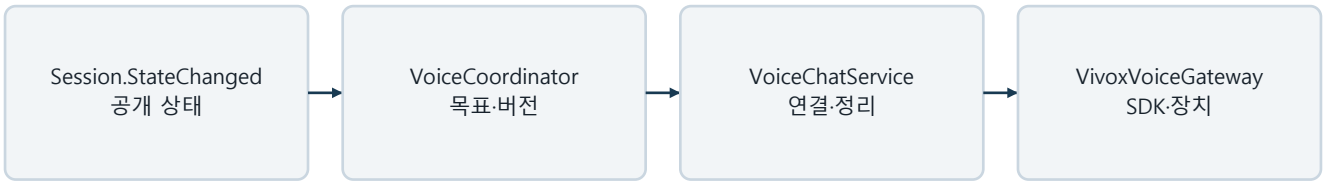
Select는 이전 전환이 끝나지 않았어도 현재 gains를 새 시작값으로 저장합니다. 1→2 도중 1번으로 돌아가도 음량이 초기값으로 뛰지 않습니다.

효과음은 Interaction.FeedbackEmitted와 Gait.Stepped를 관찰합니다. 클릭·다이얼 같은 확대 UI음과 발소리·문의 조건을 모두 합치지 않습니다. 배경음은 해금이 아니라 실제 열린 문 영역 통과를 따릅니다.

> 설계 의도 · 일반 C# 보간과 엔진 출력을 분리하고 소리마다 발동 근거를 명시합니다.

30. 음성: 게임 참가에서 SDK 연결로

음성 조정자는 공개 세션 상태만 관찰하고 게임 내부 토큰·정답 규칙은 알지 못합니다.



호출과 책임

코드 경계	읽어야 할 동작
OnSessionChanged	대상이 바뀌면 목표·버전을 갱신하고 먼저 송신을 멈춘다.
PumpAsync	한 실행 흐름에서 연결·끊기를 처리하고 최신 버전으로 반복한다.
VoiceChatService	SDK 연결·실패 보상 정리를 직렬화한다.

원문을 따라 읽기

클라 / Voice/Runtime/SessionVoiceCoordinator.cs : 36-42 · 현재 원문 발췌

```
36      desired = next;
37      version++;
38      voice.StopTransmission();
39      StartPump();
40  }
41  private void StartPump()
42  {
```

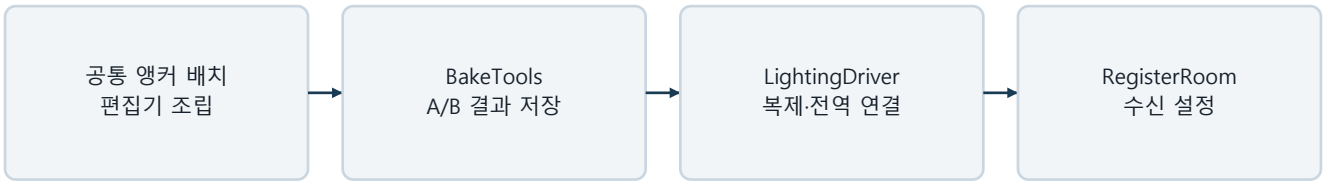
이미 Pump가 돌고 있으면 중복 시작하지 않고 버전 변경을 기존 흐름이 반영합니다. 게임 퇴장은 음성 Disconnect로 이어집니다.

취소할 수 없는 SDK 작업도 완료 뒤 보상 정리를 추적합니다. 게임 HTTP 성공은 실제 채널·마이크·장치 성공을 증명하지 않습니다.

> 설계 의도 · 관찰자·외부 어댑터와 명시적 비동기 순서를 조합합니다. 게임 서버가 음성을 중계하지 않습니다.

31. 조명: 전역 상태도 소유자를 확인한다

역할별 프로브 원본·실행 복제본·Unity 전역 연결을 분리합니다.



호출과 책임

코드 경계	읽어야 할 동작
Initialize	기존 전역을 저장하고 원본을 복제해 연결한다.
RegisterRoom / Player	BlendProbes 수신을 설정한다. 새 방마다 다시 굽지 않는다.
OnDestroy	전역이 자기 복제본일 때만 이전 상태·방향광을 복원한다.

원문을 따라 읽기

클라 / World/RoomProbeLightingDriver.cs : 81-90 · 현재 원문 발췌

```
81  bool ownsGlobal = owned != null && ReferenceEquals(LightmapSettings.lightProbes, owned);
82  if (ownsGlobal)
83  {
84      LightmapSettings.lightProbes = previous;
85      foreach (var item in directional)
86          if (item.light != null)
87              item.light.enabled = item.enabled;
88  }
89  if (owned != null)
90      Destroy(owned);
```

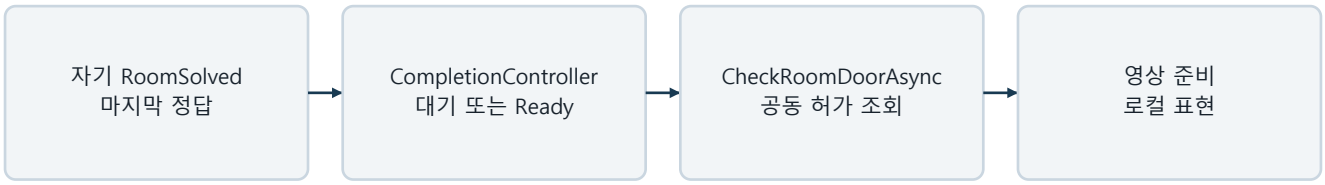
ReferenceEquals는 현재 전역 결과가 내 owned인지 확인합니다. 새 입장이 전역을 바꿨으면 옛 객체가 복원하지 않습니다.

Baked 광원 생성과 프로브 결과 적용은 다릅니다. 배치·강도·색이 바뀌면 역할 결과를 다시 굽습니다. receiveGI는 편집기 조건이고 플레이어 수신은 lightProbeUsage로 처리합니다.

> 설계 의도 · 엔진 전역도 무조건 복원하지 않고 소유권을 검사합니다. 프로브는 부드러운 빛 근사이며 정교한 벽 그림자와 같지 않습니다.

32. 공동 엔딩: 자기 정답과 양쪽 완료

마지막 문 열림·출구 통과가 아니라 양쪽 정답 조건이 기준입니다.



호출과 책임

코드 경계	읽어야 할 동작
Director.Initialize	Next=0·BothSolved 종단 방 하나와 영상 설정을 연결한다.
OnSolved / Tick	자기 마지막 정답 뒤에만 약 1초 간격 완료 확인을 시작한다.
가용성 변경	연결 종료면 Disconnected로 바꾸고 요청을 취소한다.

원문을 따라 읽기

클라 / World/EndingCompletionController.cs : 56-63 · 현재 원문 발췌

```
56      if (disposed || roomNo != finalRoomNo || Phase != EndingCompletionPhase.Playing)
57          return;
58      Phase = service.HasDoorPermission(roomNo) ? EndingCompletionPhase.Ready : EndingCompletionPhase.WaitingForPartner;
59      Changed?.Invoke();
60  }
61  public void Tick(long nowMs)
62  {
63      if (nowMs < 0)
```

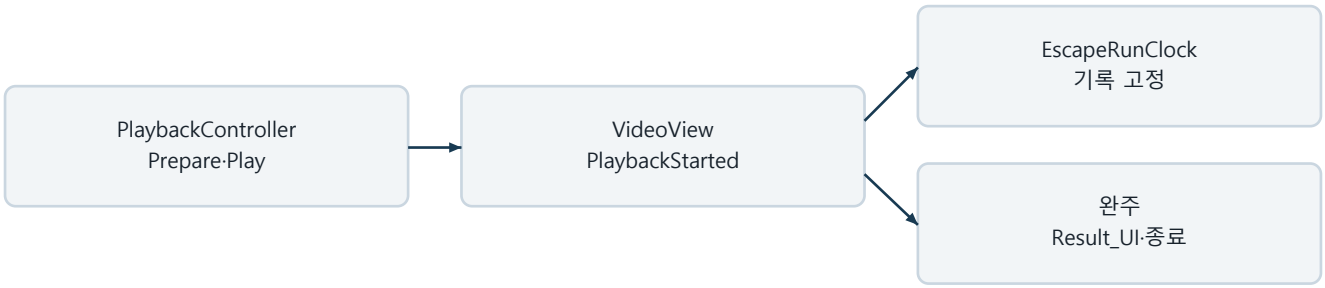
허가가 이미 있으면 Ready, 없으면 WaitingForPartner입니다. RoomSolved는 자기 성공의 로컬 이벤트이고 상대 정답은 서버 공동 허가를 조회해 확인합니다.

correct=true·doorAllowed=false는 정상 대기일 수 있습니다. 두 PC가 같은 프레임에 영상을 시작하는 시간 동기화는 별도 기능입니다.

> 설계 의도 · 공동 완료는 서버·데이터, 대기·영상은 클라가 소유합니다.

33. 영상·결과: 실제 재생 시작의 의미

Prepare 요청·재생 시작·완주·결과 표시를 구별합니다.



호출과 책임

코드 경계	읽어야 할 동작
PlaybackController	IEndingVideoPlayback으로 준비·재생·완료·실패·제한 시간을 관리한다.
EscapeRunClock	Ingame 진입 기준부터 실제 영상 시작까지 한 번만 고정한다.
Director / ResultView	완주 뒤 결과를 만들고 성공하면 영상 자원을 정리한다.

원문을 따라 읽기

클라 / World/EndingVideoDirector.cs : 152-157 · 현재 원문 발췌

```
152  private void OnVideoStarted()
153  {
154      runClock.FreezeAtVideoStart(NowMs);
155      rig.GetComponent<EscapeGame.Audio.GameFeedbackDriver>()?.SuspendForEnding();
156  }
157  private static void QuitGame()
```

영상 Prepare가 아니라 실제 PlaybackStarted에서 시간을 고정하므로 장치 준비와 상대 대기 시간을 포함합니다. 영상 재생 시간은 제외합니다.

각 PC의 로컬 기록이며 서버 순위 기록이 아닙니다. 완주 뒤 Result_UI에 탈출 시간을 채우고 게임종료 버튼을 연결합니다. 현재 Enter 키 종료 기능으로 읽지 않습니다.

> 설계 의도 · 엔진 재생과 일반 상태·시계를 분리하고 결과 UI 성공 뒤 디코더·렌더 텍스처를 해제합니다.

34. 종료: 연결·씬·UI를 따로 닫는다

성공 경로만큼 취소·퇴장·파괴의 순서를 읽어야 객체 수명을 이해할 수 있습니다.



호출과 책임

코드 경계	읽어야 할 동작
SessionClient.LeaveAsync	종료 응답 실패와 관계없이 로컬 자격을 지운다.
OnDisable / OnDestroy	활성 구독과 객체의 입력·UI·전역 상태를 정리한다.
재입장	새 연결 버전·캐시 세대·데이터 계획을 사용한다.

원문을 따라 읽기

클라 / Session/Runtime/SessionClient.cs : 141-152 · 현재 원문 발췌

```
141  var result = await SessionCommandExecutor.ExecuteAsync(current.LeaveAsync, linked.Token, diagnostic);
142  // 응답을 못 받아도 로컬은 종료하고 서버 만료가 남은 자리를 정리합니다.
143  state.Clear();
144  coordinator = null;
145  leaving = disconnected = false;
146  disconnectReason = lastBackgroundFailure = SessionFailureCode.None;
147  if (!disposed)
148      PublishState();
149  if (result.Code == SessionFailureCode.InvalidParticipant || result.Code == SessionFailureCode.SessionNotFound)
150      return SessionCommandResult.Completed();
151  if (result.Outcome == SessionCommandOutcome.Failed)
152      return SessionCommandResult.Failed(SessionFailureCode.LeaveUnconfirmed);
```

발췌는 LeaveAsync의 로컬 정리입니다. 반환 확인 실패는 LeaveUnconfirmed로 알리고 서버 만료가 남은 자리를 정리합니다. 무한히 응답을 기다리며 종료를 막지 않습니다.

Unity 객체는 C# 참조가 있어도 먼저 파괴될 수 있습니다. null 확인·중복 Dispose 보호·이벤트 해제·늦은 응답 차단을 함께 읽습니다. 예외를 catch로 숨기는 것만으로 수명이 맞아지는 것은 아닙니다.

> 설계 의도 · 생성 주체와 소유권을 넘겨받은 주체를 구별합니다. 모든 프로세스 종료가 정상 leave를 보낸다고 가정하지 않습니다.

35. 다형성·패턴·이벤트의 실제 적용

패턴 이름보다 교체 단위와 책임을 먼저 봅니다. 필요한 곳에만 다른 기법을 조합했습니다.

기법	실제 코드	이득·책임
전략	IPuzzleAnswerRule → SlotAnswerRule	판정 교체. 현재 여러 종류가 공통 규칙 공유
어댑터	입력·HTTP·IVoiceGateway IEndingVideoPlayback	외부 API 격리. 실제 장치 검사는 남음
템플릿 메서드	GameApiClientBase → SendAsync	생성 호출 틀과 Unity 전송의 실제 상속
팩토리	LocalPlayerFactory AnchorFeatureFactory	생성·합성·실패 정리
파사드	SessionClient·VoiceChatService	소비자에게 작은 기능 창구
관찰자	StateChanged·RoomLoaded MotionApplied·RoomSolved	완료 결과를 독립 소비자에 전달
상태 모델	GameSession.phase·입력·영상 Phase	허용 순서. 모두 GoF State는 아님
주입·합성	main·Bootstrapper·Initialize	의존성과 소유자 추적

35.1 구독은 순서와 수명까지 계약이다

구독 전에 상태를 읽는지, 구독 직후 재검사하는지, 상태 저장 뒤 알리는지 확인합니다. OnEnable에서 붙였다면 OnDisable에서 해제하는 경로도 읽습니다. 이벤트는 명령 재시도·네트워크 전파 장치가 아닙니다.

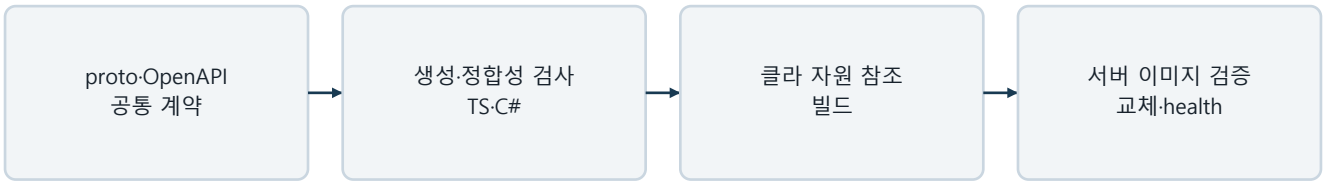
35.2 완료로 설명하지 않는 확장

Decorator·이벤트 소싱·결정적 리플레이·호스트 이전을 근거 없이 사용했다고 부르지 않습니다. P2P·전용 서버 교체에는 권한·명령 순서·상태 직렬화·시간·물리 동기화를 별도로 설계해야 합니다.

> 앞으로도 데이터 정의·가변 상태·규칙·표현·인프라를 나누고 MonoBehaviour는 엔진 생명주기와 어댑터에 집중합니다.

36. 생성·빌드·배포·로그 도구 관계

자동화의 원본과 관리 생성물, 수동 게임 구현을 구별합니다.



원본·도구	행동	자동으로 하지 않는 것
protocol/escape/v1	메시지·직렬화 생성	정답 규칙·프리팹 작성
openapi.json	경로·호출 틀·Swagger	Controller 사용 사례 구현
content-contract.json	기능명·인자·종류 공통 정의	새 Unity 기능 구현
api:sync / --apply	검사·계획 / 해시·백업 후 반영	Git 커밋·실행 파일 배포
GameContentBuilder	활성 CSV·프리팹 빌드 참조	새 콘텐츠 제작
DockerDeployment	기존 유지 중 빌드·검증 뒤 교체	무중단·자동 롤백·진행 복구

36.1 로그는 완료 결과를 관찰한다

HttpAccessLogger.observe는 finish·close 중 한 번만 기록하고 구독을 해제합니다. ILogSink는 출력 교체 경계이며 현재는 ConsoleJsonLogSink의 표준 출력입니다. 토큰·답 원문은 저장하지 않고 로그 출력 실패가 게임 요청을 뒤집지 않게 합니다.

> Docker 기본 회전은 파일당 10MB·최대 3개입니다. 앱 오류 전용 파일 저장과 다르며 CPU·메모리 제한 옵션은 현재 배포 도구에 없습니다. 클라이언트는 기존 Stable 결과 전체를 함께 배포합니다.

연결 코드: scripts/sync-game-api.cjs · scripts/docker-deploy · Editor/GameContentBuilder.cs · src/infrastructure/http/HttpAccessLogger.ts

37. 증상에서 메서드를 찾는 진단 지도

마지막으로 확정된 상태를 찾습니다. 통신 성공·생성 성공·표현 성공을 따로 확인합니다.

증상	읽을 순서	구분할 점
두 명인데 시작 안 됨	GameSession.phase → ready → IsGameplayReady → 생성 오류	join은 준비 완료가 아님
다음 방이 안 보임	해금 ID → RoomsChanged → ApplyRooms → failures·앵커	해금과 생성은 다름
답은 맞는데 문 잠김	Value / DoorAllowed → ExitRule·solved·permitted	BothSolved 대기는 정상일 수 있음
종료 중 참조 예외	OnDisable·OnDestroy·Dispose → 늦은 응답·Unity null·잠금	catch만으로 수명 해결 안 됨
배경음 안 바뀜	문 열림·영역·이전/현재 위치 → EnterRoom·fade·음원	해금만으로 전환 안 됨
조명이 안 보임	역할 결과·owned·수신 설정 → 배치 변경 후 베이크	Baked 광원 생성은 계산 아님
엔딩이 안 나옴	자기 RoomSolved → 공동 허가 → Prepare·Started·Finished	문 열림은 엔딩 조건 아님

37.1 각 검사가 증명하는 범위

일반 C#·TypeScript 검사는 규칙·주기·상태를 확인합니다. 편집기 참조 컴파일은 플레이어 전용 API 오류를 모두 검출하지 못할 수 있습니다. 실제 배포 빌드에서 양쪽 ready·7개 방·음성·영상·종료를 별도로 확인해야 합니다.

> 이번 문서 작성은 새 전체 테스트 결과가 아닙니다. 순수 조회 할당·파일 크기·GPU·전체 메모리도 다른 지표로 구분합니다. 기존 검사 기록의 날짜와 소스 버전을 함께 읽습니다.

38. 코드와 기존 문서 색인

상대 경로는 클라 Assets/3.Script와 서버 src 아래입니다. 발췌의 전체 메서드는 이 경로에서 이어 읽습니다.

기능	클라이언트	서버
입장·준비	World/GameEntryService.cs World/SessionStartGate.cs Session/SessionSceneLoadedController.cs	services/SessionService.ts models/GameSession.ts
통신·상태	Session/Runtime/SessionClient.cs Session/SessionCoordinator.cs Networking/ProtobufGameApiClient.cs	controllers/SessionController.ts infrastructure/http/ createSessionHttpServer.ts
데이터·월드	Data/CsvDataStore.cs World/RoomLoadPlan.cs World/IngameWorldDriver.cs	data/GameDataBundle.ts data/validation/CsvValidator.ts
플레이어	Player/LocalPlayerFactory.cs Player/PlayerMovementDriver.cs Player/PlayerControlState.cs	위치·입력 처리는 클라 로컬
상호작용·문	Interaction/CloseupController.cs Interaction/PuzzleAnswerController.cs World/RoomPermissionService.cs	models/PuzzleAnswerRules.ts models/RoomProgression.ts
소리·음성·조명	Audio/GameFeedbackDriver.cs Voice/Runtime/SessionVoiceCoordinator.cs World/RoomProbeLightingDriver.cs	별도 표현·외부 서비스 경계
엔딩·결과	World/EndingCompletionController.cs World/EndingVideoDirector.cs World/EndingResultView.cs	공동 완료는 RoomProgression

상세 설계는 클라 Docs/ClientCodeArchitecture.md와 서버 docs/ServerCodeArchitecture.md, 작업 규격은 RefactorConventions를 봅니다. 데이터 작성은 DesignerDataTableGuide·DataDrivenInteraction, 실기 절차는 LocalPlayTestGuide·RoomProbeLighting·GameFeedback·EndingVideo로 이어갑니다.

> 기능 추가 순서: 책임·권한 → 데이터 → 명령·상태 → 완료 이벤트 → 어댑터 → 수명 → 비용 → 실패 검사.

39. 코드를 직접 따라 읽는 연습

수정 없이도 한 명령의 입력·상태·출력을 표로 적으면 모듈 관계를 익힐 수 있습니다.

39.1 탈출하기부터 조작 허용까지

JoinRequested → Presenter.JoinAsync → EnterAsync → SessionClient.JoinAsync → 서버 join → 데이터 등록 → StartGate → EnterSceneAsync → IsGameplayReady → ready → Playing → CanPlay를 읽습니다. "자리 확보·생성·조작 허용"을 따로 적습니다.

39.2 정답부터 문과 다음 방까지

SubmitAsync → IRoomAnswerService → PermissionService → Coordinator.DecideRoomAsync → 서버 decideRoom/submit/checkDoor → snapshot 적용 → RoomsChanged → ApplyRooms를 읽습니다. value·doorAllowed·unlockedRoomIds·IsOpen·loaded를 각 열에 씁니다.

39.3 화면을 닫는 동안 응답이 온다면

닫기 → Controller.Dispose → lifetime.Cancel → 응답 후 토큰·disposed 확인 → 구독 해제 → 잠금 해제를 추적합니다. 확대 화면만 닫는 수명과 게임 연결 전체 종료를 비교합니다.

39.4 Git 저장소

클라이언트
https://github.com/lkhwcf-arch/-listen

서버
https://github.com/needAction/TSC_REST

> 기준일: 2026-10-26 현재 로컬 소스. 원격 주소는 Git origin 기준이며 공개 여부·최신 반영은 별도 확인하지 않았습니다. 원문 발췌 행 번호는 이후 수정으로 바뀔 수 있습니다.